

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

на создание автоматизированной информационной системы

«ЧистоДома»

Сайт заказа услуг клининга

Разработчик: Аньшин Сергей

Группа: ИСП-406

Статус документа: учебное техническое задание

2026

1 Общие сведения

1.1 Полное наименование системы и условное обозначение

Полное наименование: автоматизированная информационная система заказа услуг клининга «ЧистоДома». Условное обозначение: АИС «ЧистоДома».

1.2 Заказчик и разработчик

Заказчик: образовательная организация в рамках выполнения практической работы. Разработчик: Аньшин Сергей, группа ИСП-406.

1.3 Основания для разработки

- задание на практическую работу 3 по проектированию заголовков безопасности, защите cookie и созданию прототипа сайта;
- дополнительные требования к сайту клининга;
- учебные материалы по требованиям безопасности, HTTP-заголовкам и безопасности cookie;
- ГОСТ 34.602-2020 «Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы».

1.4 Плановые сроки выполнения

Разработка выполняется в течение периода, отведённого учебным планом на практическую работу. Дата завершения определяется датой передачи исходного кода, базы данных и отчётной документации преподавателю.

1.5 Источники финансирования

Работа является учебной и отдельного финансирования не предусматривает.

2 Цели и назначение создания системы

2.1 Цели создания

Цель создания АИС «ЧистоДома» — автоматизировать регистрацию клиентов, приём заявок на клининг, хранение данных о заявках и контроль их статусов. Дополнительная цель — продемонстрировать применение безопасных HTTP-заголовков и параметров cookie.

Критерии достижения цели: пользователь может зарегистрироваться и войти, создать заявку и увидеть её статус; администратор может просмотреть все заявки и изменить статус; данные сохраняются в PostgreSQL; браузер получает предусмотренные заголовки безопасности и сессионный cookie с безопасными атрибутами.

2.2 Назначение

Система предназначена для взаимодействия клиентов клининговой службы с оператором сайта. Пользователь оформляет заявку через веб-интерфейс, а администратор контролирует поступившие заявки.

3 Характеристика объекта автоматизации

Объект автоматизации — процесс приёма и сопровождения заявок на уборку жилых и офисных помещений. До автоматизации заявки могут фиксироваться вручную, что затрудняет их учёт и контроль статусов.

Система эксплуатируется как веб-приложение в современном браузере. Учебная версия запускается локально по адресу 127.0.0.1. Серверная часть взаимодействует с локальным экземпляром PostgreSQL 18.

4 Требования к автоматизированной системе

4.1 Требования к структуре системы

Система должна состоять из следующих компонентов:

- клиентская часть: HTML, CSS и JavaScript;
- серверная часть: приложение на Python;
- подсистема хранения данных: PostgreSQL;
- подсистема управления сессиями на основе cookie;
- интерфейс администратора для управления статусами заявок.

Взаимодействие браузера и сервера выполняется по HTTP в локальной учебной среде и должно выполняться по HTTPS при публикации. Обмен прикладными данными осуществляется в формате JSON.

4.2 Режимы функционирования

- обычный режим — регистрация, вход, создание и просмотр заявок;
- административный режим — просмотр всех заявок и изменение статусов;
- режим отказа — возврат пользователю общего сообщения без трассировки стека и внутренних данных сервера.

4.3 Диагностирование

Сервер должен обнаруживать ошибки входных данных, отсутствие авторизации, ошибки доступа и ошибки базы данных. Для пользователя должны выводиться краткие сообщения. Технические сведения и пароли не должны передаваться в браузер.

4.4 Перспективы развития

Допускается добавление расчёта стоимости, расписания исполнителей, уведомлений, восстановления пароля, разграничения дополнительных ролей, HTTPS reverse проху, резервного копирования и контейнерного развёртывания.

5 Требования к функциям

5.1 Просмотр сайта и поиск

- отображение услуг: поддерживающая уборка, генеральная уборка, уборка после ремонта;
- переход по основным разделам страницы;
- поиск по подготовленному списку разделов обычным строковым сравнением без исполнения регулярных выражений, HTML и JavaScript.

5.2 Регистрация пользователя

- ввод фамилии, имени, отчества, логина, пароля и адреса;
- обязательное согласие на обработку персональных данных;
- проверка ФИО: русские буквы, заглавная первая буква, допустимый дефис;
- длина каждого поля, кроме пароля и адреса, не более 20 символов; адрес не более 100 символов;
- логин содержит только латинские буквы и должен быть уникальным;

- пароль содержит не менее 8 символов, латинские буквы обоих регистров, цифру и символ из набора \$#@!;
- после успешной регистрации пользователь автоматически получает сессию.

5.3 Авторизация и завершение сессии

- проверка логина и пароля;
- создание случайного сессионного токена при успешном входе;
- удаление серверной сессии и очистка cookie при выходе;
- отказ во входе с общим сообщением при неверных данных.

5.4 Работа с заявками

- создание заявки доступно только авторизованному пользователю;
- поля заявки: вид уборки, адрес, дата и телефон;
- дата не может быть раньше текущей;
- номер телефона проверяется по формату 8(XXX)XXX-XX-XX;
- новая заявка получает статус «Новая»;
- пользователь видит только собственные заявки.

5.5 Администрирование

- администратор видит заявки всех пользователей;
- администратор может установить статус «Новая», «В работе», «Выполнена» или «Отменена»;
- право изменения статуса проверяется на сервере.

5.6 Уведомление о cookie

При первом открытии страницы должна отображаться плашка об использовании необходимых cookie. После нажатия «Понятно» отметка о закрытии хранится в localStorage. Сведения о назначении cookie должны быть доступны в отдельном разделе страницы.

6 Требования к видам обеспечения

6.1 Математическое обеспечение

Специальные математические модели не требуются. Для генерации токенов должен применяться криптографически стойкий генератор случайных значений. Для хеширования паролей применяется PBKDF2-HMAC-SHA256 со случайной солью не менее 28 символов или 32 байт и достаточным числом итераций.

6.2 Информационное обеспечение

В PostgreSQL должны использоваться таблицы:

- users — ФИО, логин, хеш пароля, адрес, роль, время согласия и дата создания;
- sessions — хеш токена, ссылка на пользователя и срок действия;
- orders — ссылка на пользователя, услуга, адрес, дата визита, телефон, статус и дата создания.

Первичные и внешние ключи должны обеспечивать целостность связей. Логин должен иметь ограничение уникальности. При удалении пользователя связанные сессии и заявки удаляются каскадно. Доступ к базе выполняется только параметризованными запросами.

6.3 Лингвистическое обеспечение

Язык интерфейса — русский. Кодировка страниц и JSON — UTF-8. Сообщения должны быть понятными пользователю и не содержать технических трассировок.

6.4 Программное обеспечение

- Python 3.14 или совместимая версия Python 3;
- драйвер psycopg 3;
- PostgreSQL 18 или совместимая актуальная версия;
- современный браузер с поддержкой Fetch API, dialog, localStorage и cookie;
- pgAdmin 4 допускается как средство администрирования базы данных.

6.5 Техническое обеспечение

Для учебной эксплуатации требуется персональный компьютер под управлением Windows 10 или Windows 11, не менее 4 ГБ оперативной памяти и свободный локальный порт 8000 для приложения и порт 5432 для PostgreSQL.

6.6 Метрологическое обеспечение

Требования к метрологическому обеспечению не предъявляются, поскольку система не выполняет измерения физических величин.

6.7 Организационное обеспечение

Пользователь самостоятельно регистрируется и создаёт заявки. Администратор контролирует заявки и статусы. Пароли администратора и базы данных должны храниться в локальном файле окружения .env, исключённом из системы контроля версий.

6.8 Методическое обеспечение

При сопровождении применяются настоящее техническое задание, инструкция README, материалы практической работы и документация PostgreSQL и Python.

7 Общие технические требования

7.1 Требования к персоналу

Для работы пользователя достаточно навыков использования браузера. Администратор должен уметь выполнять вход и изменять статус заявки. Для установки требуется специалист, способный запустить PostgreSQL, установить зависимости Python и задать переменные окружения.

7.2 Показатели назначения

- создание учётной записи при корректных данных;
- сохранение заявки и отображение её без повторного ручного ввода;
- разделение доступа пользователя и администратора;
- корректное отображение при ширине экрана около 390 пикселей;
- выдача защитных заголовков во всех ответах сервера.

7.3 Надёжность

Операции записи должны выполняться в транзакциях PostgreSQL. Ошибка базы данных не должна приводить к выдаче пользователю внутренних сведений. Просроченные сессии должны считаться недействительными. После перезапуска приложения данные пользователей и заявок должны сохраняться в базе.

7.4 Требования по безопасности

Сервер должен добавлять следующие HTTP-заголовки:

- Content-Security-Policy со значениями default-src 'self', script-src 'self', style-src 'self', img-src 'self' data:, object-src 'none', base-uri 'self', form-action 'self', frame-ancestors 'none';
- X-Content-Type-Options: nosniff;
- X-Frame-Options: DENY;
- Referrer-Policy: strict-origin-when-cross-origin;
- Permissions-Policy с запретом камеры, микрофона и геолокации;
- Cross-Origin-Opener-Policy: same-origin;
- Cache-Control: no-store;
- Strict-Transport-Security при работе по HTTPS.

Сессионный cookie должен иметь Path=/, HttpOnly, SameSite=Lax и ограниченный Max-Age. В режиме HTTPS дополнительно применяются Secure и имя с префиксом __Host-. Атрибут Domain для cookie с префиксом __Host- не задаётся.

Токен сессии должен генерироваться модулем secrets. В базе хранится SHA-256-хеш токена. Пароли хранятся только как PBKDF2-хеш со случайной солью. Сравнение хешей выполняется безопасным методом compare_digest.

Все входные данные проверяются на сервере по белому списку форматов и ограничениям длины. Изменяющие запросы принимаются только в JSON и только при совпадении Origin с адресом приложения. SQL-запросы параметризуются. Административные действия защищаются серверной проверкой роли.

7.5 Эргономика и техническая эстетика

Интерфейс должен быть простым, читаемым и адаптивным. Формы имеют подписи, сообщения выводятся рядом с соответствующим действием. Элементы не должны выходить за границы экрана шириной 390 пикселей.

7.6 Транспортабельность

Требования к физической транспортабельности не предъявляются. Исходный код переносится копированием каталога проекта и восстановлением параметров окружения.

7.7 Эксплуатация и обслуживание

Запуск выполняется командой `python server.py` после настройки DATABASE_URL. Перед публикацией необходимо настроить HTTPS, заменить учебные тексты согласия и политик на утверждённые документы оператора персональных данных и организовать резервное копирование PostgreSQL.

7.8 Защита информации от несанкционированного доступа

Неавторизованный пользователь не получает список заявок и не может создать заявку. Авторизованный пользователь видит только собственные записи. Изменять статусы может только администратор. Секреты запрещено включать в исходный код и публиковать в репозитории.

7.9 Сохранность информации при авариях

Для рабочей эксплуатации требуется регулярное резервное копирование базы PostgreSQL. После сбоя приложение должно запускаться без изменения схемы и использовать сохранённые данные. В учебной версии резервное копирование выполняется средствами PostgreSQL или pgAdmin.

7.10 Защита от внешних воздействий

Специальные требования к физическим воздействиям не предъявляются. Приложение должно работать в пределах условий эксплуатации персонального компьютера и локальной сети.

7.11 Патентная чистота

В системе используются общедоступные технологии и библиотеки. При распространении необходимо соблюдать лицензии Python, PostgreSQL и pycorg.

7.12 Стандартизация и унификация

Имена API-маршрутов, таблиц и полей должны быть единообразными. Даты передаются в формате ISO 8601. HTTP-коды ответа должны соответствовать результату операции: 200, 201, 400, 401, 403, 404, 409, 415 и 500.

8 Состав и содержание работ

- 1) Анализ предметной области и требований практической работы.
- 2) Проектирование структуры страницы, ролей и сценариев работы.
- 3) Проектирование таблиц PostgreSQL и связей.
- 4) Разработка интерфейса и адаптивного оформления.
- 5) Разработка серверных маршрутов регистрации, входа, выхода и заявок.
- 6) Настройка сессий, cookie и заголовков безопасности.
- 7) Проверка функций пользователя и администратора.
- 8) Проверка заголовков в Network, cookie в Application и данных в pgAdmin.
- 9) Подготовка технической и отчётной документации.

9 Порядок разработки

Разработка выполняется последовательно от структуры данных к серверной логике и интерфейсу. Изменения проверяются локально. Конфиденциальные параметры помещаются в .env. Исходный код и документация передаются как единый каталог проекта.

10 Порядок контроля и приёмки

Приёмка проводится функциональной проверкой по следующим сценариям:

- регистрация с корректными и некорректными данными;
- отказ при повторном логине;
- вход с правильным и неправильным паролем;
- создание заявки после входа и отказ без входа;
- просмотр пользователем только собственных заявок;
- изменение статуса администратором и отказ обычному пользователю;
- удаление сессии при выходе;
- наличие заголовков безопасности в HTTP-ответе;
- наличие HttpOnly, SameSite=Lax, Path=/ и Max-Age у cookie;
- адаптивное отображение при ширине 390 пикселей;
- сохранение данных в таблицах users, sessions и orders.

Система принимается при успешном прохождении перечисленных проверок и отсутствии критических ошибок.

11 Подготовка объекта к вводу в действие

- 1) Установить PostgreSQL и создать базу cleaning.
- 2) Установить Python и зависимости из requirements.txt.
- 3) Создать файл .env с DATABASE_URL, ADMIN_LOGIN и ADMIN_PASSWORD.
- 4) Запустить сервер командой python server.py.
- 5) Открыть сайт в браузере и проверить регистрацию, вход и заявку.
- 6) Для сетевого размещения настроить HTTPS и reverse proxy.
- 7) Разместить утверждённые документы по обработке персональных данных и cookie.

12 Требования к документированию

В комплект документации входят: исходный код, requirements.txt, README с инструкцией запуска, пример файла окружения без секретов, схема базы данных или описание таблиц, техническое задание и отчёт о практической работе со скриншотами проверки.

13 Источники разработки

- ГОСТ 34.602-2020;
- задание на практическую работу 3;
- дополнительные требования для задания по клинингу;
- учебные презентации «Требования безопасности», «Заголовки безопасности» и «Обеспечение безопасности cookie»;
- документация Python, PostgreSQL и psycopg.